

Linux 应用开发手册

Revision History

Draft Date	Revision No.	Description
2025/04/03	V1.0	1. 初始版本。

目 录

前 言.....	4
1 Linux 常用开发案例.....	5
1.1 led_flash 案例.....	5
1.1.1 案例功能.....	5
1.1.2 操作说明.....	5
1.1.3 关键代码.....	6
1.2 key_demo 案例.....	7
1.2.1 案例功能.....	7
1.2.2 操作说明.....	8
1.2.3 关键代码.....	9
1.3 can_echo 案例.....	11
1.3.1 案例功能.....	11
1.3.2 操作说明.....	11
1.3.3 关键代码.....	16
1.4 tcp_udp 案例.....	17
1.4.1 案例功能.....	17
1.4.2 操作说明.....	19
1.4.3 关键代码.....	21
1.5 uart_rw 案例.....	21
1.5.1 案例功能.....	21
1.5.2 操作说明.....	22
1.5.3 关键代码.....	29
1.6 devmem_rw 案例.....	34
1.6.1 案例说明.....	34
1.6.2 案例测试.....	35
1.6.3 案例编译（ARM 端）.....	38
1.6.4 关键代码（ARM 端）.....	39

因我们的存在，让嵌入式应用更简单

2 Python 开发案例40

2.1 led_flash 案例41

2.1.1 案例功能41

2.1.2 操作说明41

2.1.3 关键代码42

2.2 key_demo 案例43

2.2.1 案例功能43

2.2.2 操作说明44

2.2.3 关键代码46

更多帮助.....48

前言

本文档涉及的开发案例位于产品资料“4-软件资料\Demo\”路径下的 base-demos 和 python-demos 目录。

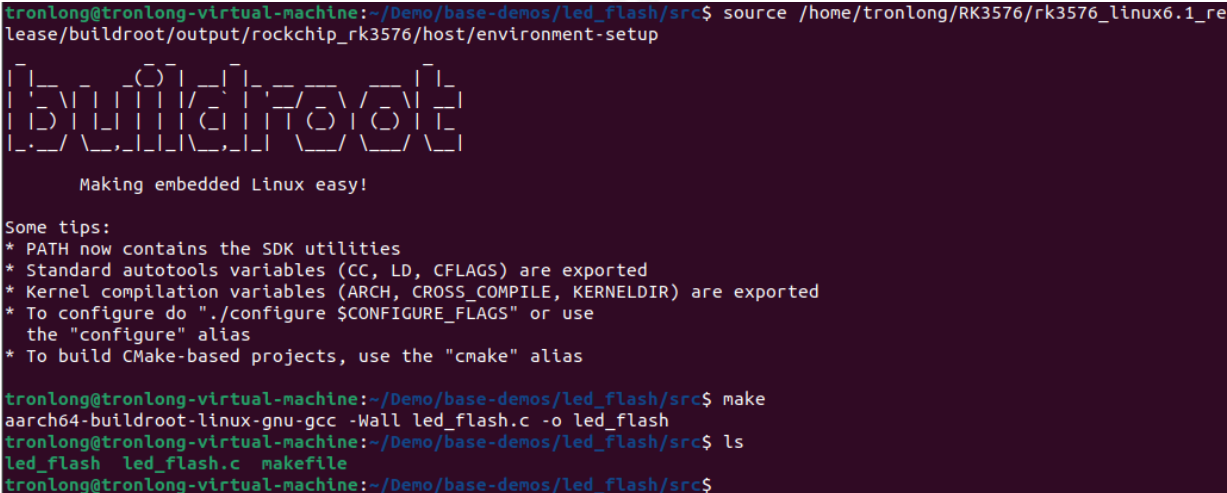
base-demos 目录存放 Linux 常用开发案例，案例 bin 目录存放可执行文件，案例 src 目录存放源码。python-demos 目录存放 Python 开发案例，案例脚本文件无需编译，可基于我司提供的文件系统直接运行。

如需重新编译 Linux 常用开发案例，请先参考《Linux 系统使用手册》文档正确安装 LinuxSDK，构建适配评估板的交叉编译工具链。然后将对应案例 src 目录拷贝至 Ubuntu 工作目录下，在 src 目录执行如下命令配置交叉编译工具链，并执行 make 命令进行案例编译。编译完成后，将在当前目录下生成可执行文件。

备注：LinuxSDK 版本号以实际情况为准。

```
Host# source /home/tronlong/RK3576/rk3576_linux6.1_release/buildroot/output/rockchip_p_rk3576/host/environment-setup
```

```
Host# make
```



```
tronlong@tronlong-virtual-machine:~/Demo/base-demos/led_flash/src$ source /home/tronlong/RK3576/rk3576_linux6.1_release/buildroot/output/rockchip_rk3576/host/environment-setup

[Tronlong]

Making embedded Linux easy!

Some tips:
* PATH now contains the SDK utilities
* Standard autotools variables (CC, LD, CFLAGS) are exported
* Kernel compilation variables (ARCH, CROSS_COMPILE, KERNELDIR) are exported
* To configure do "./configure $CONFIGURE_FLAGS" or use the "configure" alias
* To build CMake-based projects, use the "cmake" alias

tronlong@tronlong-virtual-machine:~/Demo/base-demos/led_flash/src$ make
aarch64-buildroot-linux-gnu-gcc -Wall led_flash.c -o led_flash
tronlong@tronlong-virtual-machine:~/Demo/base-demos/led_flash/src$ ls
led_flash led_flash.c makefile
tronlong@tronlong-virtual-machine:~/Demo/base-demos/led_flash/src$
```

图 1

因我们的存在，让嵌入式应用更简单

1 Linux 常用开发案例

1.1 led_flash 案例

1.1.1 案例功能

本案例通过向评估底板用户可编程指示灯 LED 设备节点反复交替写入 1、0 数值，实现 LED 闪烁效果。LED 点亮与熄灭时间均为 0.5s。

程序流程如下图所示。

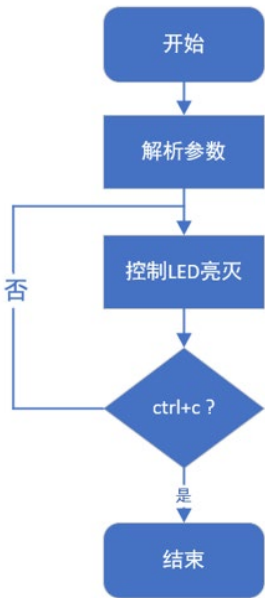


图 2

LED 设备节点为"/sys/class/leds/user-ledX/"目录下的 brightness。

```
root@RK3576-Tronlong:~# ls /sys/class/leds/
disk heartbeat mmc0:: user-led0 user-led1
root@RK3576-Tronlong:~# ls /sys/class/leds/user-led0
brightness device max_brightness power subsystem trigger uevent
root@RK3576-Tronlong:~# ls /sys/class/leds/user-led1
brightness device max_brightness power subsystem trigger uevent
root@RK3576-Tronlong:~#
```

图 3

1.1.2 操作说明

将本案例 bin 目录下的可执行程序 led_flash 拷贝至评估板文件系统，并在可执行程序

因我们的存在，让嵌入式应用更简单

所在目录执行如下命令运行程序，即可看到评估底板 LED 以 0.5s 的时间间隔进行闪烁。

同时，串口终端打印系统全部 LED 设备信息和程序当前控制的 LED 设备信息。

Target# ./led_flash -h

Target# ./led_flash -n 2

```
root@RK3576-Tronlong:~# ./led_flash -h
usage: led_flash [options]

Options:
  -n | --number          Number of LEDs, range of 1 to 4
  -v | --version          Display version information
  -h | --help            Show help content

root@RK3576-Tronlong:~# ./led_flash -n 2

System leds :
/sys/class/leds/disk
/sys/class/leds/heartbeat
/sys/class/leds/mmc0:
/sys/class/leds/user-led0
/sys/class/leds/user-led1

Flashing leds :
/sys/class/leds/user-led0
/sys/class/leds/user-led1
```

图 4

1.1.3 关键代码

- (1) 预定义 LED 数组。程序由此数组获取 LED 信息，数组信息必须为系统已有 LED 信息，否则程序运行报错。

```
17  /* User-operable LEDs */
18  static char *g_leds[] = {
19      "/sys/class/leds/user-led0",
20      "/sys/class/leds/user-led1",
21      "/sys/class/leds/user-led2",
22      "/sys/class/leds/user-led3"
23  };
```

图 5

- (2) LED 亮灭操作和时间间隔。

```
128 while (!g_quit) {
129     /* Turn on LEDs */
130     for (i = 0; i < num; i++) {
131         /* Set the LED brightness value to 1 to turn on the led */
132         snprintf(cmd, 64, "echo 1 > %s/brightness", g_leds[i]);
133         if (system(cmd) != 0) {
134             fprintf(stderr, "Error: Failed to turn on %s\n", g_leds[i]);
135             exit(EXIT_FAILURE);
136         }
137     }
138
139     /* Keep the LEDs on for 500 ms */
140     usleep(500 * 1000);
141
142     /* Turn off LEDs */
143     for (i = 0; i < num; i++) {
144         /* Set the LED brightness value to 0 to turn off the LED */
145         snprintf(cmd, 64, "echo 0 > %s/brightness", g_leds[i]);
146         if (system(cmd) != 0) {
147             fprintf(stderr, "Error: Failed to turn off %s\n", g_leds[i]);
148             exit(EXIT_FAILURE);
149         }
150     }
151
152     /* Keep the LEDs off for 500 ms */
153     usleep(500 * 1000);
154 }
```

图 6

1.2 key_demo 案例

1.2.1 案例功能

本案例通过监听用户输入按键 USER1(KEY4)、USER2(KEY5)的状态，检测按键事件。案例将获取按键事件后进行按键键值匹配，再进行事件处理，程序流程如下图所示。

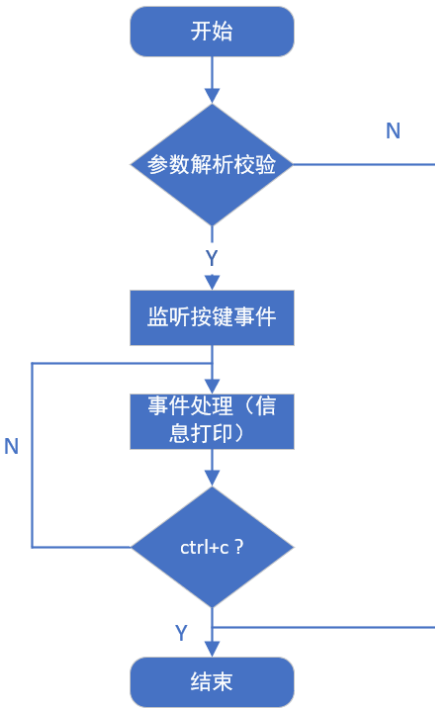


图 7

因我们的存在，让嵌入式应用更简单

1.2.2 操作说明

本次测试以用户输入按键 USER1(KEY4)示例进行测试，如需测试用户输入按键 USER2(KEY5)，请修改对应的设备节点。

表 1

用户输入按键	按键事件号
USER1(KEY4)	/dev/input/event5
USER2(KEY5)	/dev/input/event8

执行如下命令，查看 USER1(KEY4)按键对应的设备节点。

Target# cat /proc/bus/input/devices

```
root@RK3576-Tronlong:~# cat /proc/bus/input/devices
I: Bus=0019 Vendor=0000 Product=0000 Version=0000
N: Name="rk805_pwrkey"
P: Phys=rk805_pwrkey/input0
S: Sysfs=/devices/platform/2ac40000.i2c/i2c-1/1-0023/rk805-pwrkey.2.auto/input/input0
U: Uniq=
H: Handlers=kbd event0
B: PROP=0
B: EV=3
B: KEY=10000000000000 0

I: Bus=0019 Vendor=0001 Product=0001 Version=0100
N: Name="headset-keys"
P: Phys=headset-keys/input0
S: Sysfs=/devices/platform/es8388-sound/input/input1
U: Uniq=
H: Handlers=kbd event1
B: PROP=0
B: EV=3
B: KEY=10000000000 0 0
```

图 8

```
I: Bus=0019 Vendor=0001 Product=0001 Version=0100
N: Name="adc-keys2"
P: Phys=adc-keys/input0
S: Sysfs=/devices/platform/adc-keys2/input/input5
U: Uniq=
H: Handlers=kbd event5
B: PROP=0
B: EV=3
B: KEY=100000 0 0

I: Bus=0000 Vendor=0000 Product=0000 Version=0000
N: Name="rockchip-dp0 rockchip-dp0"
P: Phys=ALSA
S: Sysfs=/devices/platform/dp0-sound/sound/card1/input6
U: Uniq=
H: Handlers=event6
B: PROP=0
B: EV=21
B: SW=40
```

图 9

将本案例 bin 目录下的可执行程序 key_demo 拷贝至评估板文件系统，在可执行程序所在目录执行如下命令运行程序，串口终端将打印提示信息。再按下评估板用户输入按键 USER1(KEY4)，程序将检测到按键事件，并打印按键状态信息。

Target# ./key_demo -h

Target# ./key_demo -d /dev/input/event5

```
root@RK3576-Tronlong:~# ./key_demo -h
Usage: key_demo [options]

Options:
-d | --device          Device
-v | --version         Display version information
-h | --help           Show help content

root@RK3576-Tronlong:~# ./key_demo -d /dev/input/event5
Please press the key to test.
User key pressed!
User key pressed!
User key pressed!
```

图 10

1.2.3 关键代码

(1) 定义按键。

```
18
19 typedef enum { KEY_CODE_NONE = 0, KEY_CODE_USER1, KEY_CODE_USER2 } KeyCode;
20
```

图 11

(2) 监听按键事件。

```
76 static int check_button_pressed(int fd) {
77     assert(fd >= 0);
78
79     /* wait button being pressed or released. */
80     fd_set input;
81     FD_ZERO(&input);
82     FD_SET(fd, &input);
83     int ret = select(fd + 1, &input, NULL, NULL, NULL);
84     if (ret < 0) {
85         printf("%s", strerror(errno));
86         return -1;
87     }
88
89     /* read event */
90     struct input_event buf;
91     if (read(fd, &buf, sizeof(struct input_event)) < 0) {
92         printf("%s", strerror(errno));
93         return -1;
94     }
95
96     /* Check the input_event value */
97     switch (buf.code) {
98     case KEY_PROG1:
99         /* 1: pressed; 0: released */
100         if (buf.value == 1)
101             return KEY_CODE_USER1;
102         break;
103     case KEY_PROG2:
104         if (buf.value == 1)
105             return KEY_CODE_USER2;
106         break;
107     default:
108         return KEY_CODE_NONE;
109         break;
110     }
111
112     return KEY_CODE_NONE;
113 }
```

图 12

(3) 循环监听。

```
154 while (!g_quit) {
155     int key_code = check_button_pressed(fd);
156     if (key_code < 0)
157         continue;
158
159     switch (key_code) {
160     case KEY_CODE_USER1:
161         printf("Key user1 pressed!\n");
162         break;
163     case KEY_CODE_USER2:
164         printf("Key user2 pressed!\n");
165         break;
166     default:
167         break;
168     }
169 }
```

图 13

因我们的存在，让嵌入式应用更简单

1.3 can_echo 案例

1.3.1 案例功能

本案例实现通过绑定一个 CAN 总线接口用于接收数据，当 CAN 端口接收到数据后，将数据重新发送到 CAN 总线接口。

程序流程如下图所示。

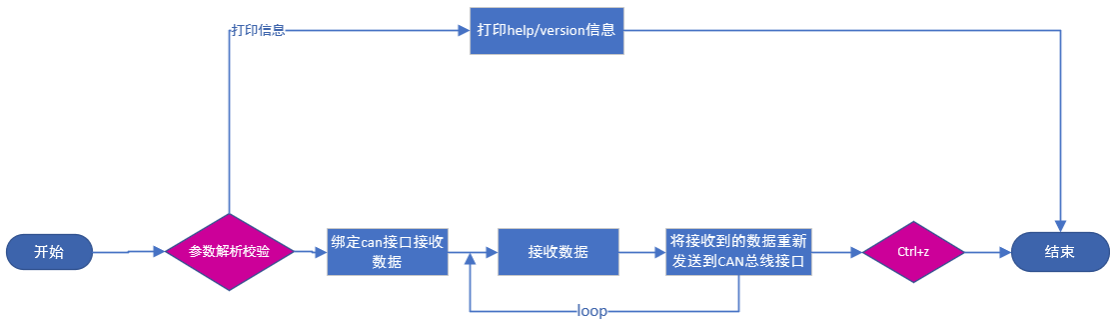


图 14

1.3.2 操作说明

请使用 USB-CAN 分析仪（型号：广成科技 USBCAN-II FD）连接评估板 CAN-FD1 或 CAN-FD2 任一接口至 PC 机 USB 接口，硬件连接如下图所示。

本次测试以 CAN-FD1 接口（设备节点为：can0）为例进行演示。如需测试 CAN-FD2 接口，请将设备节点修改为 can1。

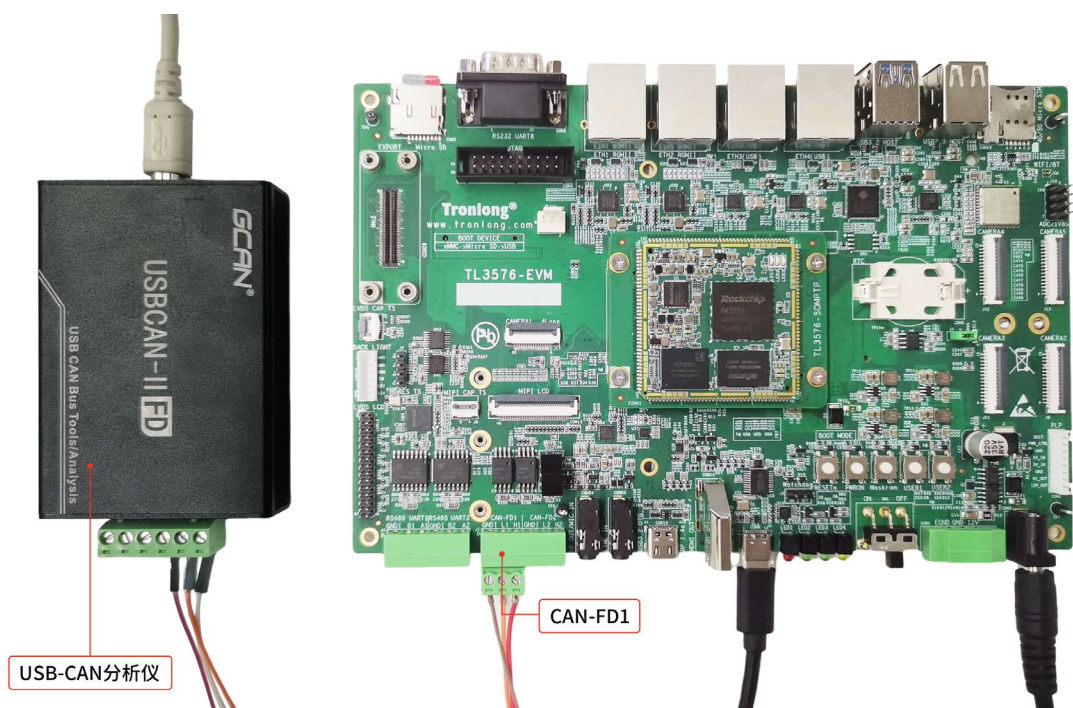


图 15

评估板 CAN-FD1 接口与 USB-CAN 分析仪的连接关系如下表所示。

表 2

评估板	USB-CAN 分析仪
CAN-FD1 接口 H1 端子	H
CAN-FD1 接口 L1 端子	L
GNDI 端子	G

请参照《调试工具安装》文档安装 GCAN Tools 调试软件。在 PC 端打开 GCANTools 软件，选择设备类型，然后点击“打开设备”。根据 USB-CAN 分析仪实际硬件连接，选择 CAN1 或 CAN2。等待软件检测到设备后，将设置波特率为"1000K"，数据波特率设置为 5000K，然后点击“确定”，如下图所示。

因我们的存在，让嵌入式应用更简单



图 16

进入 GCAN Tools 软件发送界面，取消"CAN FD"选项，USB-CAN 分析仪连接成功后将会显示 Connected 状态，如下图所示。

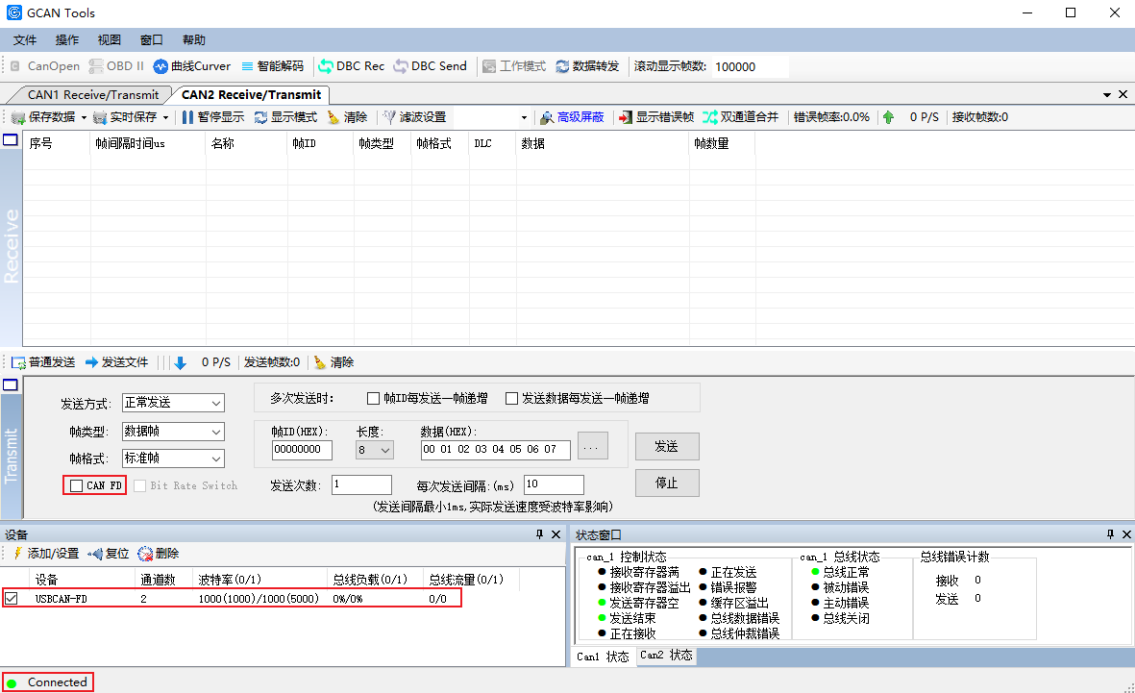


图 17

评估板启动进入评估板文件系统，执行如下命令配置 CAN0 总线波特率为 1Mbps，配置数据波特率为 5Mbps，并启动 CAN0 总线。

因我们的存在，让嵌入式应用更简单

Target# ip link set can0 down

Target# ip link set can0 type can bitrate 1000000 dbitrte 5000000 fd on

Target# ip link set can0 up

```
root@RK3576-Tronlong:~# ip link set can0 down
root@RK3576-Tronlong:~# ip link set can0 type can bitrate 1000000 dbitrte 5000000 fd on
root@RK3576-Tronlong:~# [ 58.037996] rk3576_canfd 2ac00000.can can0: bitrate error 0.3%
[ 58.038007] rk3576_canfd 2ac00000.can can0: bitrate error 1.0%
root@RK3576-Tronlong:~# ip link set can0 up
[ 94.935401] IPV6: ADDRCONF(NETDEV_CHANGE): can0: link becomes ready
root@RK3576-Tronlong:~#
```

图 18

将本案例 bin 目录下的可执行程序 can_echo 拷贝至评估板文件系统，进入可执行程序所在目录，执行如下命令查看程序参数信息。

Target# ./can_echo --help

```
root@RK3576-Tronlong:~# ls can_echo
can_echo
root@RK3576-Tronlong:~# ./can_echo --help
Usage: can_echo <can-interface> [<can-interface-out>] [Options]

Send all messages received on <can-interface> to <can-interface-out>
If <can-interface-out> is omitted, then <can_interface> is used for sending

Options:
-f, --family=FAMILY    Protocol family (default PF_CAN = 29)
-t, --type=TYPE        Socket type, see man 2 socket (default SOCK_RAW = 3)
-p, --protocol=PROTO   CAN protocol (default CAN_RAW = 1)
-v, --verbose          be verbose
-h, --help             this help
--version              print version information and exit
root@RK3576-Tronlong:~#
```

图 19

执行如下命令绑定 CAN-FD1 接口，并接收由 GCAN Tools 发出的数据，然后将接收到的数据重新发送出去。在 GCAN Tools 中输入数据并点击发送按钮，可看到有两帧数据，一帧是发送数据，另一帧是接收数据。

Target# ./can_echo -v can0

```
root@RK3576-Tronlong:~# ./can_echo -v can0
interface-in = can0, interface-out = can0, family = 29, type = 3, proto = 1
```

图 20

因我们的存在，让嵌入式应用更简单

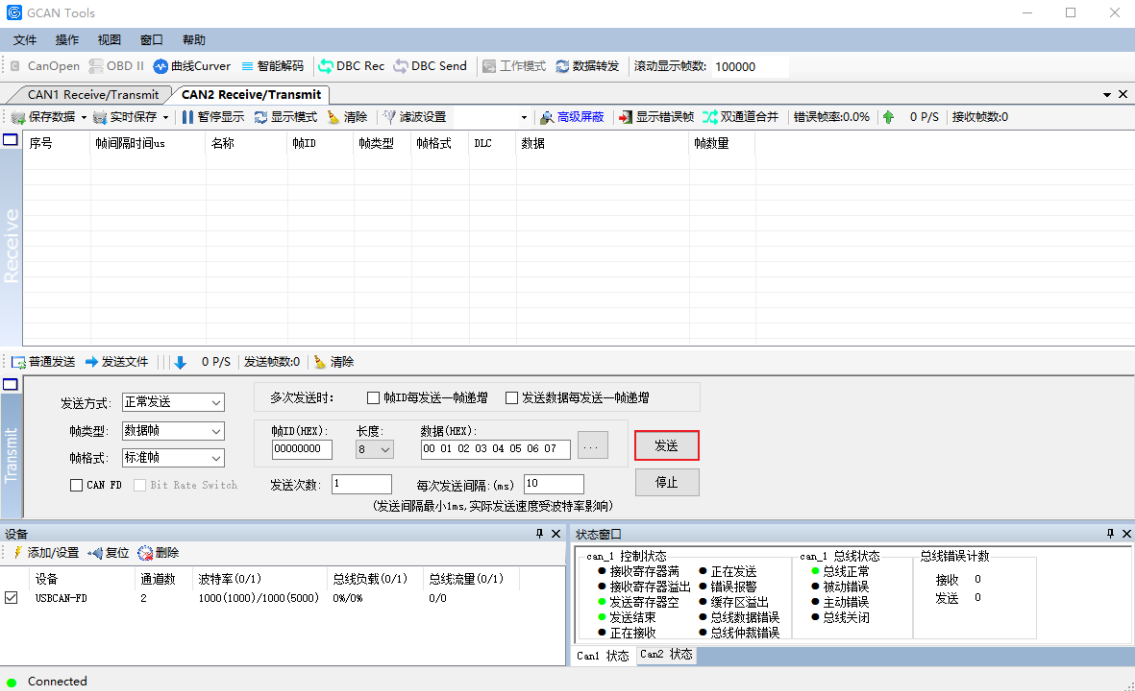


图 21

```
root@RK3576-Tronlong:~# ./can_echo -v can0
interface-in = can0, interface-out = can0, family = 29, type = 3, proto = 1
0000: [8] 00 01 02 03 04 05 06 07
```

图 22

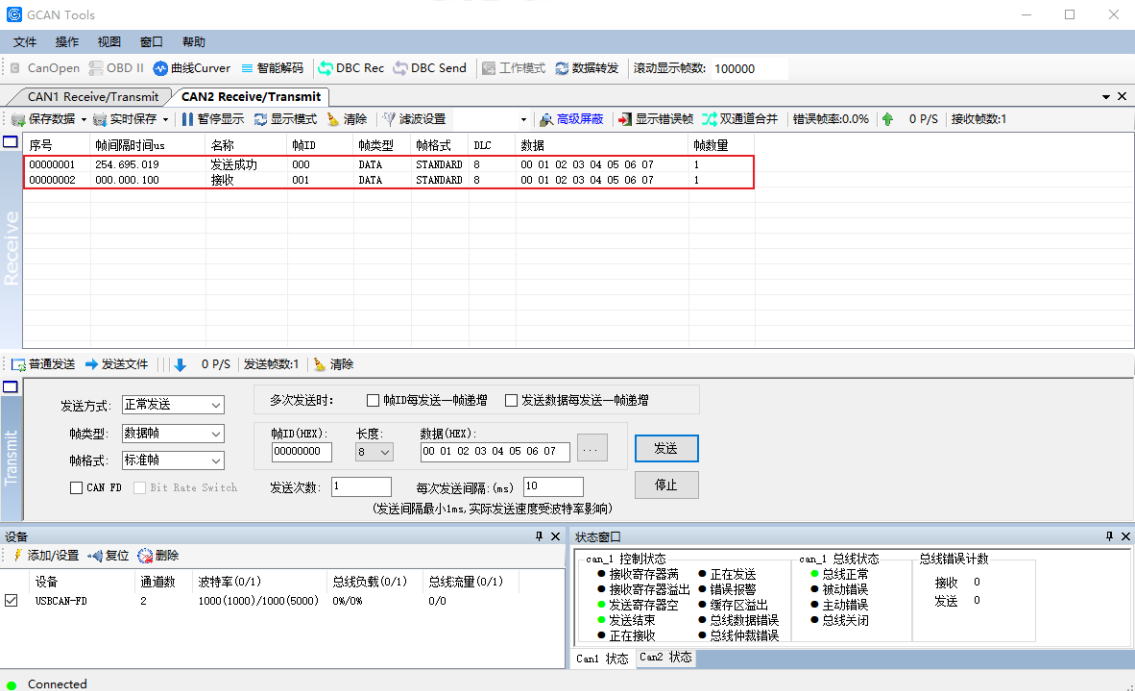


图 23

因我们的存在，让嵌入式应用更简单

可按"Ctrl + Z"暂停程序，并执行如下命令退出程序。

Target# killall -9 can_echo

```
root@RK3576-Tronlong:~# ./can_echo -v can0
interface-in = can0, interface-out = can0, family = 29, type = 3, proto = 1
0000: [8] 00 01 02 03 04 05 06 07

[1]+  Stopped                  ./can_echo -v can0
root@RK3576-Tronlong:~# killall -9 can_echo
[1]+  Killed                   ./can_echo -v can0
root@RK3576-Tronlong:~#
```

图 24

1.3.3 关键代码

(1) 使用 socket 监听 CAN 接口。

```
128 | for (i = 0; i <= out; i++) {
129 |     if ((s[i] = socket(family, type, proto)) < 0) {
130 |         perror("socket");
131 |         return 1;
132 |     }
133 |
134 |     addr[i].can_family = family;
135 |     strcpy(ifr[i].ifr_name, intf_name[i]);
136 |     ioctl(s[i], SIOCGIFINDEX, &ifr[i]);
137 |     addr[i].can_ifindex = ifr[i].ifr_ifindex;
138 |
139 |     if (bind(s[i], (struct sockaddr *)&addr[i], sizeof(addr)) < 0) {
140 |         perror("bind");
141 |         return 1;
142 |     }
143 | }
```

图 25

(2) 将从 CAN 接口接收到的数据重新发送出去。


```
145 while (running) {
146     if ((nbytes = read(s[0], &frame, sizeof(frame))) < 0) {
147         perror("read");
148         return 1;
149     }
150     if (verbose) {
151         printf("%04x: ", frame.can_id);
152         if (frame.can_id & CAN_RTR_FLAG) {
153             printf("remote request");
154         } else {
155             printf("[%d]", frame.can_dlc);
156             for (i = 0; i < frame.can_dlc; i++) {
157                 printf(" %02x", frame.data[i]);
158             }
159         }
160         printf("\n");
161     }
162     frame.can_id++;
163     write(s[out], &frame, sizeof(frame));
164 }
```

图 26

1.4 tcp_udp 案例

1.4.1 案例功能

本案例主要实现客户端(client)与服务端(server)的文本数据相互收发功能。本案例包含 4 个程序：

- (1) tcp_server: TCP 服务端测试程序。
- (2) tcp_client: TCP 客户端测试程序。
- (3) udp_server: UDP 服务端测试程序。
- (4) udp_client: UDP 客户端测试程序。

程序流程如下图所示。

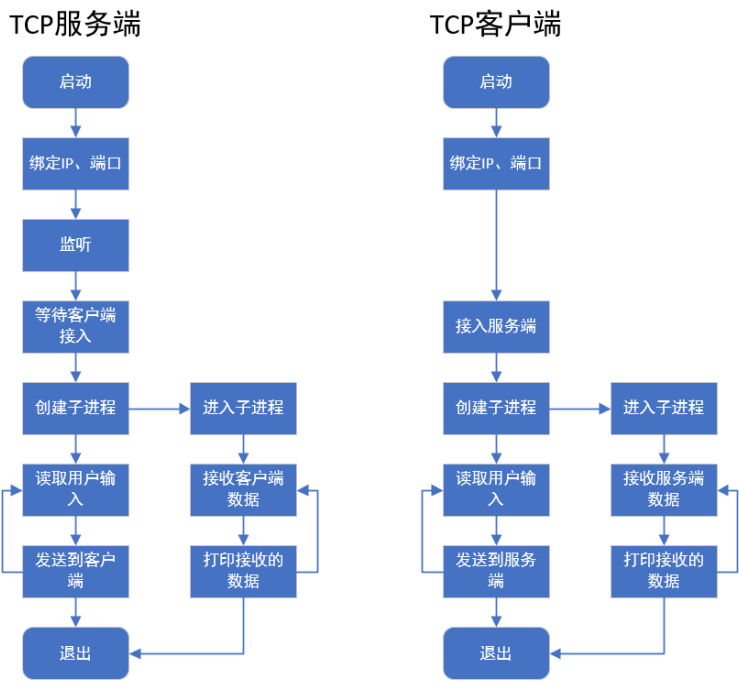


图 27 TCP 通信

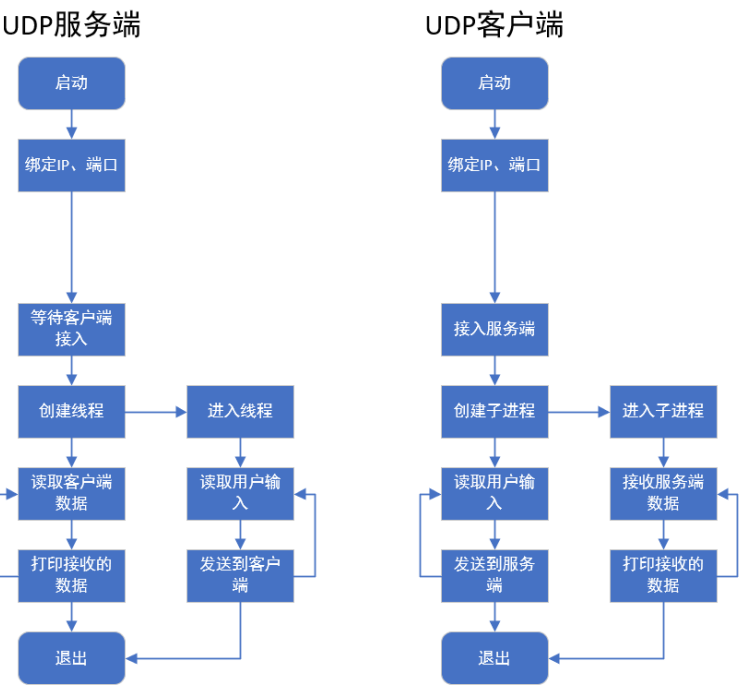


图 28 UDP 通信

服务端和客户端程序均可在评估板、PC 机 Ubuntu 系统上运行。本章节采用评估板本地回环测试，即服务端与客户端程序均在评估板上运行，通过 127.0.0.1 进行本地回环测试，无需经过路由器。案例 bin 目录下的 4 个文件均为 ARM 端可执行程序。

因我们的存在，让嵌入式应用更简单

1.4.2 操作说明

将本案例 bin 目录下的 4 个可执行程序拷贝至评估板文件系统。

在 Ubuntu 中执行如下命令使用 OpenSSH 登录评估板文件系统，如下图所示。

Host# `sudo ssh root@192.168.13.27` //192.168.13.27 为评估板 IP 地址，请根据实际情况修改

```
tronlong@tronlong-virtual-machine:~$ sudo ssh root@192.168.13.27
[sudo] password for tronlong:
root@RK3576-Tronlong:~# ls
tcp_client tcp_server udp_client udp_server
root@RK3576-Tronlong:~#
```

图 29

在评估板系统中执行如下命令，启用本地回环接口。

Target# `ifconfig lo up`

```
root@RK3576-Tronlong:~# ifconfig lo up
root@RK3576-Tronlong:~#
```

图 30

(1) TCP 通信测试

在可执行程序所在目录执行如下命令，分别在评估板运行 TCP 服务端和客户端程序。

2233 为服务端程序指定的端口号，客户端程序端口号参数需与服务端程序一致。

Target# `./tcp_server 2233` //TCP 服务端命令

Host# `./tcp_client 127.0.0.1 2233` //TCP 客户端命令

程序执行后，客户端将会连接服务端。在服务端(Target)输入字符串"Tronlong"，按下回车键即可在客户端显示对应内容。在客户端(Host)输入字符串"Hello Tronlong"，按下回车键即可在服务端显示对应内容，如下图所示。

```
root@RK3576-Tronlong:~# ls
tcp_client tcp_server udp_client udp_server
root@RK3576-Tronlong:~# ./tcp_server 2233
waiting for client connection...
Client accepted : 127.0.0.1:49496
Tronlong
[Recv from client 0.0.0.0:2233] : Hello Tronlong
```

图 31 TCP 服务端

因我们的存在，让嵌入式应用更简单

```
root@RK3576-Tronlong:~# ls
tcp_client tcp_server udp_client udp_server
root@RK3576-Tronlong:~# ./tcp_client 127.0.0.1 2233
connect server(IP:127.0.0.1 Port: 2233).
[Recv from server 127.0.0.1:2233] : Tronlong
Hello Tronlong
```

图 32 TCP 客户端

(2) UDP 通信测试

在可执行程序所在目录执行如下命令，分别在评估板运行 UDP 服务端和客户端程序。
2233 为服务端程序指定的端口号，客户端程序端口号参数需与服务端程序一致。

Target# ./udp_server 2233 //UDP 服务端命令

Host# ./udp_client 127.0.0.1 2233 //UDP 客户端命令

程序执行后，客户端将不会自动连接服务端。服务端在收到客户端信息前无法得知客户端的存在，因此需要客户端先向服务端发送信息。

在客户端(Host)输入字符串"Tronlong"，按下回车键即可在服务端显示对应内容。在服务端(Target)输入字符串"Hello Tronlong"，按下回车键即可在客户端显示对应内容，如下图所示。

```
root@RK3576-Tronlong:~# ls
tcp_client tcp_server udp_client udp_server
root@RK3576-Tronlong:~# ./udp_server 2233
waiting for client connection
[Recv from client 127.0.0.1:47804] : Tronlong
Hello Tronlong
```

图 33 UDP 服务端

```
root@RK3576-Tronlong:~# ls
tcp_client tcp_server udp_client udp_server
root@RK3576-Tronlong:~# ./udp_client 127.0.0.1 2233
Tronlong
[Recv from server 127.0.0.1:2233] : Hello Tronlong
```

图 34 UDP 客户端

如需在 PC 机 Ubuntu 系统运行服务端或客户端程序，请将案例 src 源码目录拷贝至 Ubuntu 工作目录。进入源码目录后执行 make 命令，即可在当前目录下生成 x86 端可执

行程序。在不同终端运行服务端或客户端程序的命令类似，但 127.0.0.1 需使用服务器 IP 地址替代。

Host# make clean

Host# make CC=gcc

```
tronlong@tronlong-virtual-machine:~/Demo/base-demos/tcp_udp/src$ ls
makefile tcp_client.c tcp_server.c udp_client.c udp_server.c
tronlong@tronlong-virtual-machine:~/Demo/base-demos/tcp_udp/src$ make clean
rm -f tcp_client tcp_server udp_client udp_server *.o *.d *.d.*
tronlong@tronlong-virtual-machine:~/Demo/base-demos/tcp_udp/src$ make CC=gcc
gcc -c tcp_client.c -o tcp_client.o -Wall -g -O2
gcc -o tcp_client tcp_client.o -lpthread
gcc -c tcp_server.c -o tcp_server.o -Wall -g -O2
gcc -o tcp_server tcp_server.o -lpthread
gcc -c udp_client.c -o udp_client.o -Wall -g -O2
gcc -o udp_client udp_client.o -lpthread
gcc -c udp_server.c -o udp_server.o -Wall -g -O2
gcc -o udp_server udp_server.o -lpthread
tronlong@tronlong-virtual-machine:~/Demo/base-demos/tcp_udp/src$ file tcp_client
tcp_client: ELF 64-bit LSB pie executable, x86-64, version 1 (SYSV), dynamically linked, interpreter /lib64/ld-linux-x86-64.so.2, BuildID[sha1]=80f5fc0e8b2d5779de6a9f50cd38b2e18634f78f, for GNU/Linux 3.2.0, with debug_info, not stripped
tronlong@tronlong-virtual-machine:~/Demo/base-demos/tcp_udp/src$
```

图 35

1.4.3 关键代码

以 TCP 通信程序为例。

(1) tcp_client.c

注意源码中的 struct sockaddr_in、socket、connect、fgets、send、recv 等数据结构和系统调用的使用。

(2) tcp_server.c

注意源码中的 struct sockaddr_in、socket、connect、fgets、send、recv、bind、listen、accept 等数据结构和系统调用的使用。

1.5 uart_rw 案例

1.5.1 案例功能

本案例实现评估板串口读写功能。

程序流程如下图所示。串口初始化时将设置波特率、模式、数据位、停止位等参数。通过设备文件描述符对串口进行读写操作。

因我们的存在，让嵌入式应用更简单

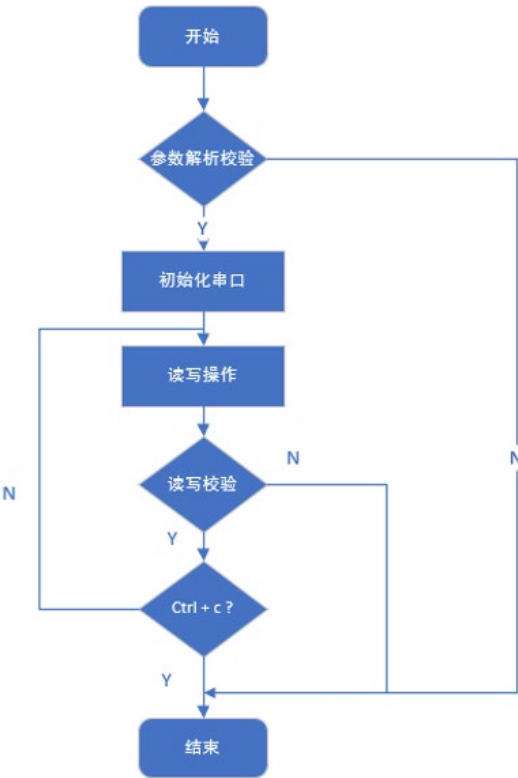


图 36 程序流程图

本案例支持的串口以及串口对应设备节点，如下表所示。

表 3

串口	设备节点
RS232 UART8	ttyS8
RS485 UART1	ttyS1
RS485 UART2	ttyS2

1.5.2 操作说明

请通过 RS232 交叉串口母母线、USB 转 RS232 公头串口线，将评估板的 RS232 UART8 调试串口连接至 PC 机的 USB 接口，如下图所示。

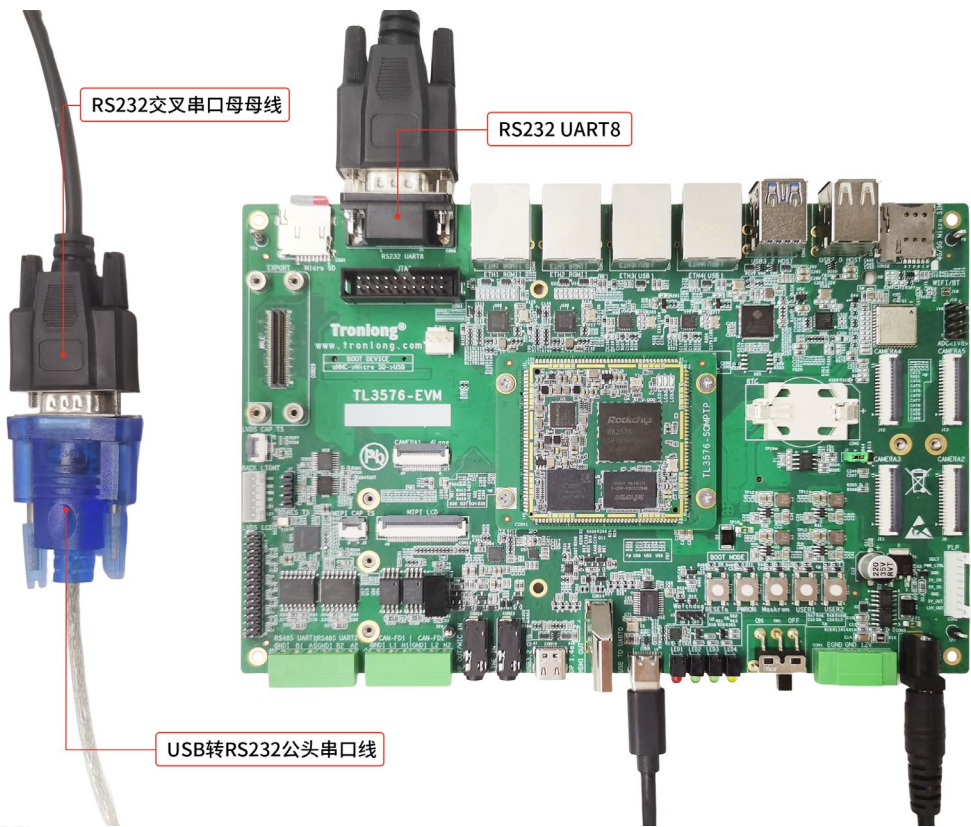


图 37

将本案例 bin 目录下的可执行程序 `uart_rw` 拷贝至评估板文件系统，并在该可执行程序所在目录执行如下命令，查看程序参数信息。

Target# `./uart_rw -h`

```
root@RK3576-Tronlong:~# ls uart_rw
uart_rw
root@RK3576-Tronlong:~# ./uart_rw -h
Usage: ./uart_rw [options]

options:
-d --device      Device such as '/dev/ttyS0'
-r --read        Read
-w --write       Write
-l --loopback    loopback test
-s --size        Read size
-v --version     Display version information
-h --help        Show help content
e.g. ./uart_rw -d /dev/ttyS1 -r -s 256
     ./uart_rw -d /dev/ttyS1 -w -s 1024
     ./uart_rw -d /dev/ttyS1 -l -s 1024
root@RK3576-Tronlong:~#
```

图 38

(1) RS232 UART8 测试

a) 评估板接收

打开设备管理器，本次测试中调试串口 USB TO UART0 端口号为 COM11，RS232 UART8

因我们的存在，让嵌入式应用更简单

串口的端口号为 COM12，如下图所示。



图 39

打开调试串口终端，COM11 波特率设为 115200，8N1，无校验位。COM12 波特率设为 115200，8N1，无校验位，并建立串口连接。评估板调试串口终端执行如下命令，评估板等待接收上位机发送的数据。

备注：评估板启动后首次执行程序 `uart_rw`，会出现打印信息"`of_dma_request_slave_channel...`"，不影响正常使用，请忽略。

Target# `./uart_rw -d /dev/ttyS8 -r -s 8`

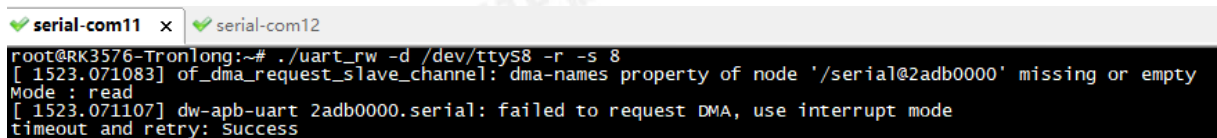


图 40

在 RS232 UART8 串口终端点击"`View -> Command (Chat) Window`"，选中并打开"`Command (Chat) Window`"窗口。

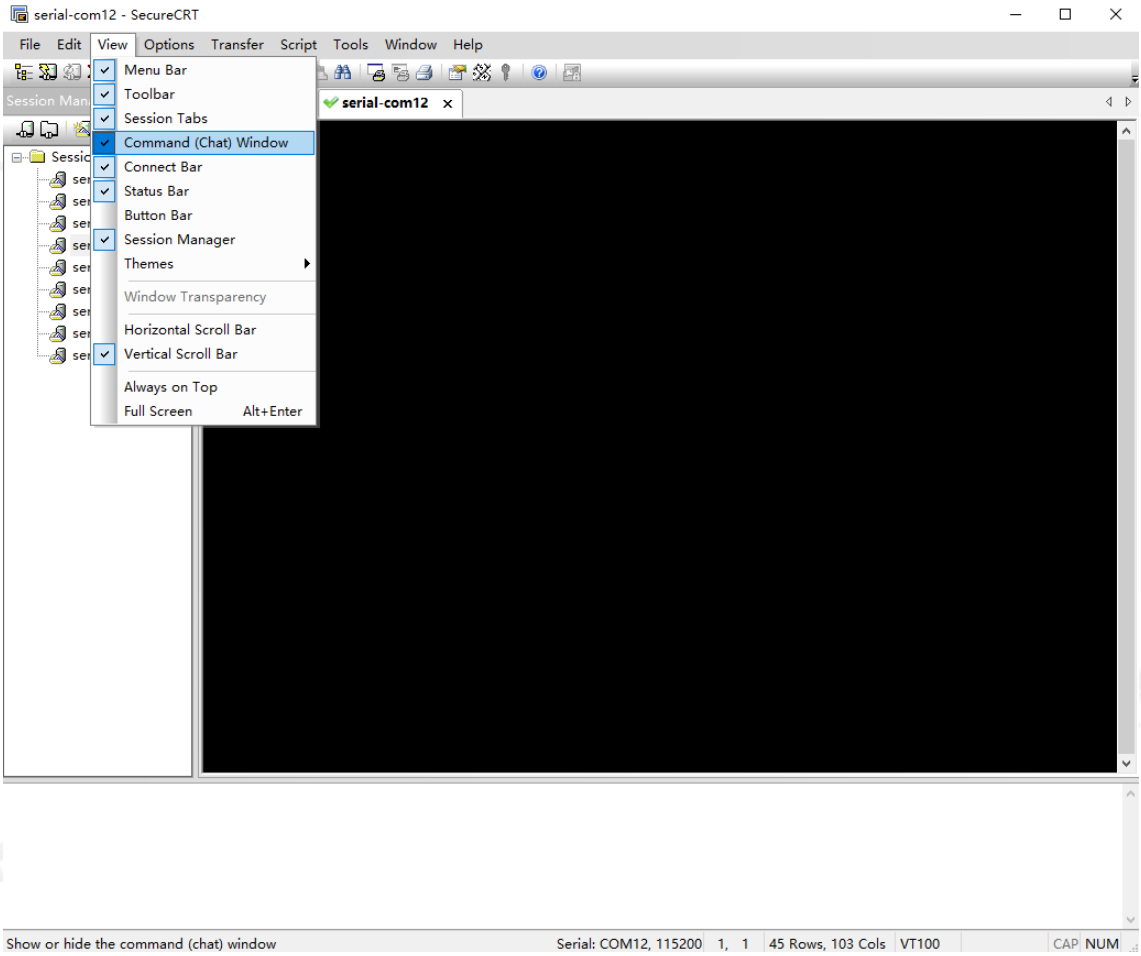


图 41

在窗口中输入数据，例如"Tronlong"，再按回车键。

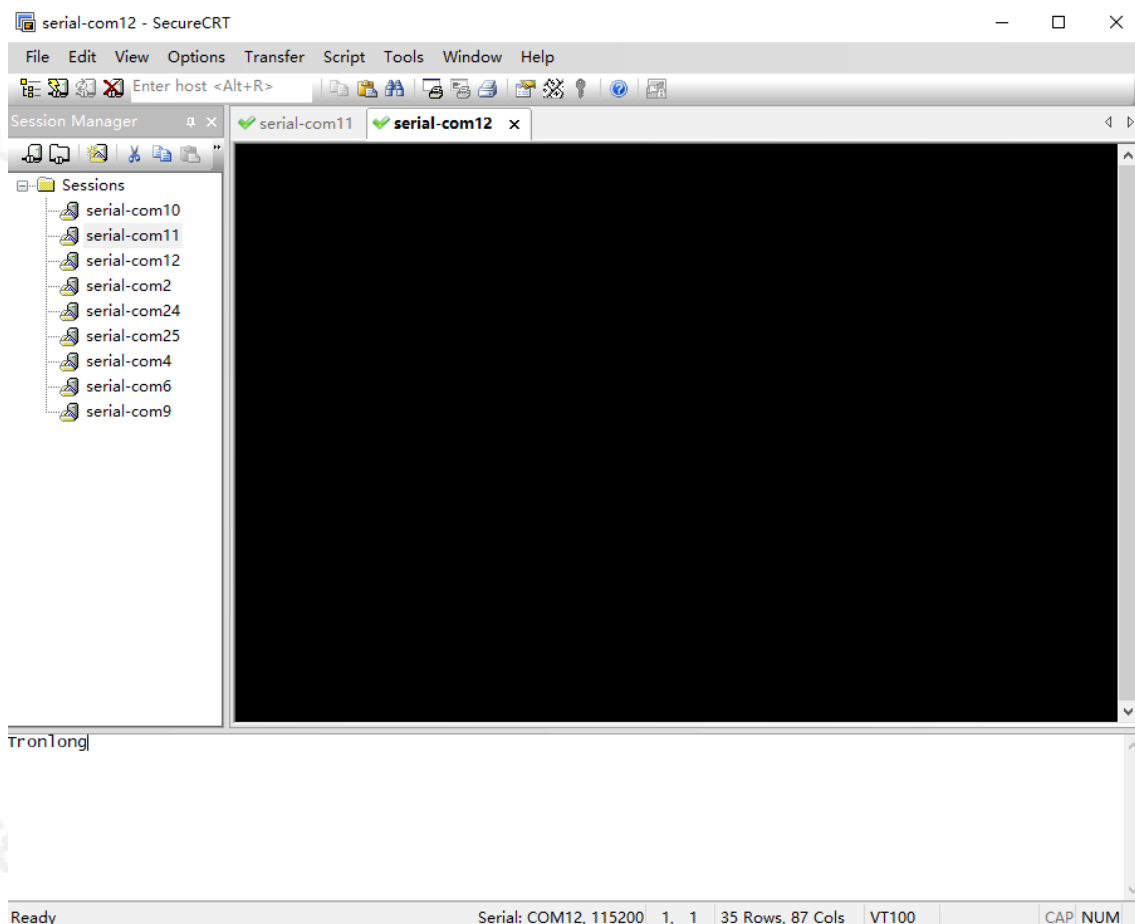


图 42

评估板调试串口终端将会打印接收到的数据。

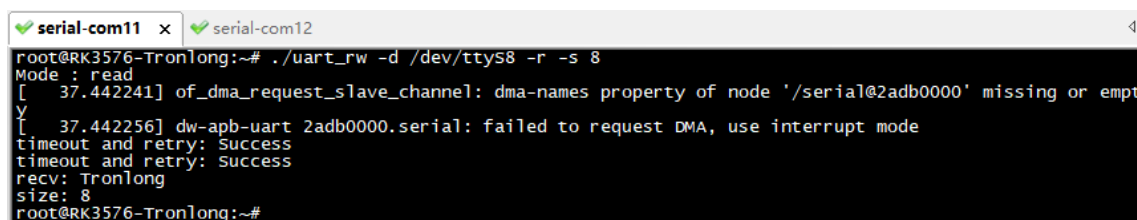


图 43

b) 评估板发送

执行如下命令，评估板通过 RS232 UART8 串口发送数据至上位机，数据已在程序中定义。

Target# ./uart_rw -d /dev/ttyS8 -w -s 8

因我们的存在，让嵌入式应用更简单

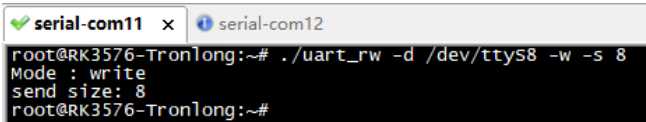


图 44

RS232 UART8 串口终端将会打印接收到的数据，如下图所示。



图 45

(2) RS485 串口测试

使用 RS232 转 RS485 模块、USB 转 RS232 公头串口线，将评估板的 RS485 UART1 或 RS485 UART2 串口连接至 PC 机的 USB 接口，如下图所示。

RS232 转 RS485 模块与评估板 RS485 串口连接方法及设备节点如下表所示。

表 4

	RS232 转 RS485 模块 485+端子	RS232 转 RS485 模块 485- 端子	RS232 转 RS485 模块 GND 端子
RS485 UART1(ttyS1)	A1 端子	B1 端子	GNDI
RS485 UART2(ttyS2)	A2 端子	B2 端子	GNDI

本章节以 RS485 UART1 串口(设备节点为: ttyS1)为例进行演示。如需测试 RS485 UART2 串口，请将设备节点修改为 ttyS2。

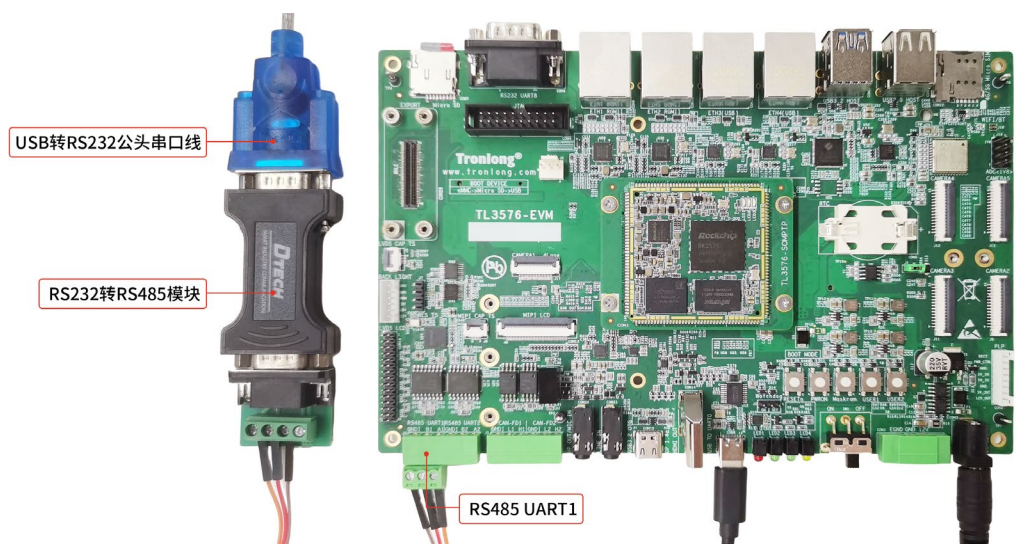


图 46

a) 评估板接收

执行如下命令，进行 RS485 UART1 串口数据接收操作。RS485 UART1 串口评估板接收测试步骤与现象与 RS232 UART8 类似，命令中的设备节点由 `ttyS8` 改成 `ttyS1` 即可。

Target# `./uart_rw -d /dev/ttyS1 -r -s 8` //读操作

```
serial-com11 x serial-com12
root@RK3576-Tronlong:~# ./uart_rw -d /dev/ttyS1 -r -s 8
Mode : read
[ 89.164089] of_dma_request_slave_channel: dma-names property of node '/serial@27310000' missing or empty
[ 89.164113] dw-apb-uart 27310000.serial: failed to request DMA, use interrupt mode
timeout and retry: Success
recv: Tronlong
size: 8
root@RK3576-Tronlong:~#
```

图 47

b) 评估板发送

执行如下命令，进行 RS485 UART1 串口数据接收操作。

Target# `./uart_rw -d /dev/ttyS1 -w -s 8` //写操作

```
serial-com10 x serial-com11
root@RK3576-Tronlong:~# ./uart_rw -d /dev/ttyS1 -w -s 8
Mode : write
send size: 8
root@RK3576-Tronlong:~#
```

图 48

因我们的存在，让嵌入式应用更简单



图 49

1.5.3 关键代码

(1) 串口初始化函数。

```
90 int init_serial(int *fd, const char *dev) {
91     struct termios opt;
92
93     /* open serial device */
94     if ((*fd = open(dev, O_RDWR)) < 0) {
95         perror("open()");
96         return -1;
97     }
98
99     /* define termois */
100    if (tcgetattr(*fd, &opt) < 0) {
101        perror("tcgetattr()");
102        return -1;
103    }
104
105    opt.c_lflag  &= ~(ICANON | ECHO | ECHOE | ISIG);
106    opt.c_oflag  &= ~OPOST;
107
108    /* Character length, make sure to screen out this bit before setting the data bit */
109    opt.c_cflag &= ~CSIZE;
110
111    /* No hardware flow control */
112    opt.c_cflag &= ~CRISCTS;
113
114    /* 8-bit data length */
115    opt.c_cflag |= CS8;
116
117    /* 1-bit stop bit */
118    opt.c_cflag &= ~CSTOPB;
119
120    /* No parity bit */
121    opt.c_iflag |= IGNPAR;
```

图 50

```
123      /* Output mode */
124      opt.c_oflag = 0;
125
126      /* No active terminal mode */
127      opt.c_lflag = 0;
128
129      /* Input baud rate */
130      if (cfsetispeed(&opt, B115200) < 0)
131          return -1;
132
133      /* Output baud rate */
134      if (cfsetospeed(&opt, B115200) < 0)
135          return -1;
136
137      /* Overflow data can be received, but not read */
138      if (tcflush(*fd, TCIFLUSH) < 0)
139          return -1;
140
141      if (tcsetattr(*fd, TCSANOW, &opt) < 0)
142          return -1;
143
144      return 0;
145  }
```

图 51

(2) 串口读写函数。

```
147  int serial_write(int *fd, const char *data, size_t size) {
148      int ret = write(*fd, data, size);
149      if (ret < 0) {
150          perror("write");
151          tcflush(*fd, TCOFLUSH);
152      }
153
154      return ret;
155  }
```

图 52


```
157 int serial_read(int *fd, char *data, size_t size) {
158     size_t read_left = size;
159     size_t read_size = 0;
160     char *read_ptr = data;
161     struct timeval timeout = {5, 0};
162
163     memset(data, 0, size);
164
165     fd_set rfds;
166     while (!g_quit) {
167         FD_ZERO(&rfds);
168         FD_SET(*fd, &rfds);
169         timeout.tv_sec = 5;
170         timeout.tv_usec = 0;
171
172         if (read_left == 0)
173             break;
174
175         switch (select(*fd+1, &rfds, NULL, NULL, &timeout)) {
176             case -1:
177                 perror("select()");
178                 break;
179             case 0:
180                 perror("timeout and retry");
181                 break;
182             default:
183                 if (FD_ISSET(*fd, &rfds)) {
184                     read_size = read(*fd, read_ptr, read_left);
185                     if (read_size == 0)
186                         break;
187
188                     read_ptr += read_size;
189                     read_left -= read_size;
190                 }
191             }
192         }
193
194     return read_size;
195 }
```

图 53

```
197 int run_read_mode(char *dev, size_t size) {
198     char *buf = NULL;
199     int fd = -1;
200     int ret = -1;
201
202     ret = init_serial(&fd, dev);
203     if (ret < 0) {
204         close(fd);
205         return -1;
206     }
207
208     printf("Mode : read\n");
209     if (size <= 0) {
210         printf("Error : Incorrect size settings\n");
211         exit(INADEQUATE_CONDITIONS);
212     }
213
214     buf = (char*)malloc(size + 1);
215     buf[size] = '\0';
216     ret = serial_read(&fd, buf, size);
217     printf("recv: %s\nsize: %d\n", buf, ret);
218
219     free(buf);
220     return 0;
221 }
```

图 54

因我们的存在，让嵌入式应用更简单

```
223 int run_write_mode(char *dev, size_t size) {
224     int fd = -1;
225     int ret = -1;
226
227     ret = init_serial(&fd, dev);
228     if (ret < 0) {
229         close(fd);
230         return -1;
231     }
232
233     printf("Mode : write\n");
234     if (size <= 0) {
235         printf("Error : Incorrect size settings\n");
236         exit(INADEQUATE_CONDITIONS);
237     }
238
239     int i = 0;
240     char context;
241     size_t write_size = 0;
242     while (!g_quit) {
243         if (i > 7)
244             i = 0;
245         context = (char)('0' + i);
246
247         write_size += serial_write(&fd, &context, sizeof(context));
248         i++;
249
250         if (size == write_size)
251             break;
252     }
253
254     printf("send size: %zd\n", write_size);
255     return 0;
256 }
```

图 55

(3) 回环测试。

```
258 int run_loopback_test(char *dev, size_t size) {
259     int fd;
260     int ret;
261     size_t buf_size;
262     int serial_buf_size;
263
264     ret = init_serial(&fd, dev);
265     if (ret < 0)
266         return -1;
267
268     printf("Start uart loopback testing.\n");
269
270     /* Serial port buffer size generally defaults to 2k - 4k */
271     char *write_buf = (char*)malloc(size);
272     char *read_buf = (char*)malloc(size);
273
274     buf_size = size;
275     while (buf_size > 0)
276     {
277         if(buf_size > 1024) {
278             serial_buf_size = 1024;
279         } else {
280             serial_buf_size = buf_size;
281         }
282
283         // Generate random data to write.
284         memset(write_buf, rand() % 26 + 65, serial_buf_size);
285         memset(read_buf, 0, serial_buf_size);
286     }
```

图 56

因我们的存在，让嵌入式应用更简单

```
287         ret = serial_write(&fd, write_buf, serial_buf_size);
288
289
290         /* delay > 1024 / 115200 * 1000000 */
291         usleep(90000);
292
293         ret = serial_read(&fd, read_buf, serial_buf_size);
294
295
296         ret = memcmp(read_buf, write_buf, serial_buf_size);
297         if (ret != 0) {
298             printf("Result : Test failed\n");
299             goto release;
300         }
301
302         buf_size -= 1024;
303     }
304     printf("send size: %zd\n", size);
305     printf("recv size: %zd\n", size);
306     printf("Result : Test pass\n");
307
308     release:
309     free(write_buf);
310     free(read_buf);
311     close (fd);
312     if (ret != 0) {
313         return NOPASS_CONDITIONS;
314     } else {
315         return 0;
316     }
317 }
```

图 57

(4) 相关逻辑。

```
371     switch (mode) {
372     case READ:
373         if(run_read_mode(dev, size) < 0) {
374             return INADEQUATE_CONDITIONS;
375         }
376         break;
377
378     case WRITE:
379         if(run_write_mode(dev, size) < 0) {
380             return INADEQUATE_CONDITIONS;
381         }
382         break;
383
384     case LOOPBACK:
385         ret = run_loopback_test(dev, size);
386         if(ret < 0) {
387             return INADEQUATE_CONDITIONS;
388         } else if(ret == NOPASS_CONDITIONS) {
389             return NOPASS_CONDITIONS;
390         }
391         break;
392     default:
393         break;
394     }
395
396     return 0;
397 }
```

图 58

因我们的存在，让嵌入式应用更简单

1.6 devmem_rw 案例

1.6.1 案例说明

本案例实现通过 PCIe 总线对外部设备进行数据读写测试，同时校验数据和计算读写速率，程序流程如下图所示。

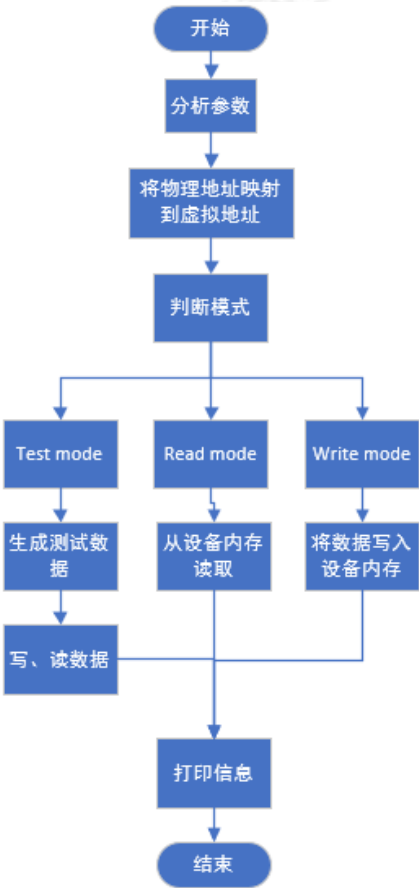


图 59

案例相关资料位于产品资料“4-软件资料\Demo\base-demos\devmem_rw\”目录下，具体说明如下：

表 5

目录		说明
arm	bin	ARM 可执行程序
	src	应用程序源码

因我们的存在，让嵌入式应用更简单

fpga	bin	FPGA 可执行程序
	project	FPGA 工程源码 (备注: 源码仅用于测试评估, 暂无案例说明)

程序流程说明:

- a) ARM 端通过 PCIe 总线将数据写入 FPGA BRAM;
- b) ARM 端通过 PCIe 总线从 FPGA BRAM 读取数据;
- c) 判断写入与读取数据的正确性, 并计算读写速率。

程序原理说明:

ARM 端:

- a) 使用 mmap 函数对物理地址进行转换;
- b) 可根据指定的访问操作类型按字节(Byte)、按半字(Halfword)进行访问。

1.6.2 案例测试

请使用 M.2 Key M 转 PCI-E 4X 转接板 (品牌: 华高智, 型号: PCI-E 4X TO M.2 M) 将创龙科技 TL3576-EVM 评估板 M.2 PCIe NVMe 接口与创龙科技 TLA7-EVM 评估板 PCIe 接口连接, 硬件连接如下图所示。

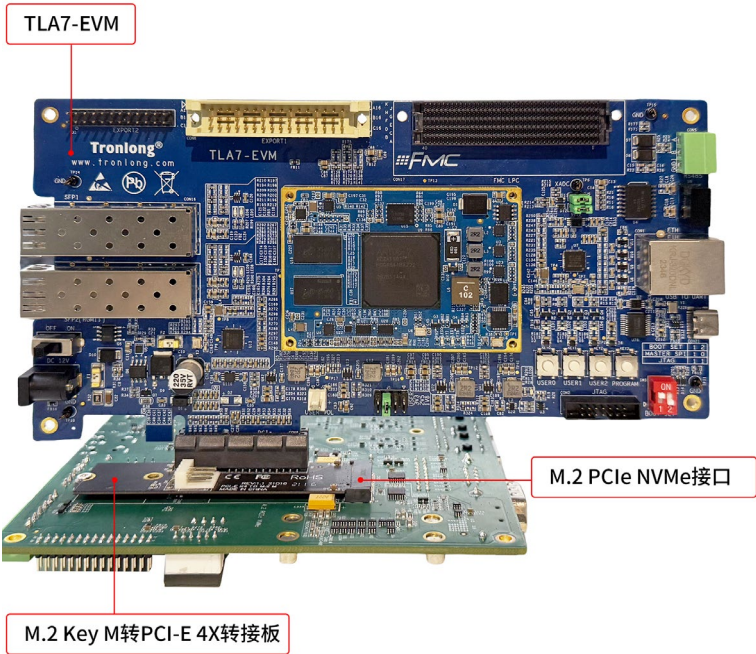


图 60

因我们的存在, 让嵌入式应用更简单

请将创龙科技 TLA7-EVM 评估板上电启动，通过下载器加载运行案例"fpga\bin\"目录下的可执行程序。该程序主要实现 PCIe Endpoint 功能，采用 PCIe 同源时钟模式。

备注：为确保 Linux 端可识别成功 PCIe EP 设备，请将 TLA7-EVM 评估板上电，等待 Done 指示灯(LED4)亮起后，再将 TL3576-EVM 评估板上电。

请将案例"arm\bin\"目录下的可执行程序 devmem_rw 拷贝至文件系统任意目录下。进入评估板文件系统，在可执行程序所在目录执行如下命令查看程序参数。

Target# ./devmem_rw -h

参数解析：

-t: 表示测试模式。先写后读，并检验数据的准确性。

-r: 读模式，从参数"-a <address>"指定的内存空间地址处，读出参数"-s <size>"指定长度的数据（以字节为单位）。

-w: 写模式，从参数"-f <file>"指定的文件获取数据，写入参数"-a <address>"指定的内存空间地址。

-a <address>: 指定内存空间地址。

-s <size>: 指定数据的长度，以字节为单位。

-o: 以字节或半字方式访问内存空间。

-f <file>: 指定文件。用于写模式测试。

```
root@RK3576-Tronlong:~# ./devmem_rw -h
Usage: ./devmem_rw [options]
Options:
  -t           Testing mode, write and then read and check consistence.
  -r           Read mode, read <size> bytes from <address>.
  -w           Write mode, write <file> to <address>.
  -a <address> Memory start address.
  -s <size>    Memory size or read data size.
  -o <byte | halfword> Access operation type.
  -f <file>    File to write.
  -v | --version
  -h | --help
e.g.
./devmem_rw -t -a 0x660000 -s 1024 -o halfword
./devmem_rw -r -a 0x660000 -s 1024 -o halfword
./devmem_rw -w -a 0x660000 -s 1024 -o halfword -f ./data.dat
root@RK3576-Tronlong:~#
```

图 61

进入评估板文件系统，执行如下命令，查看 PCIe EP 设备的连接状态。

Target# lspci -s 01:00.0 -vv

因我们的存在，让嵌入式应用更简单


```

root@RK3576-Tronlong:~# lspci -s 01:00.0 -vv
01:00.0 Serial controller: Xilinx Corporation Device 7021 (prog-if 01 [16450])
Subsystem: Xilinx Corporation Device 0007
Control: I/O- Mem- BusMaster- SpecCycle- MemWINV- VGASnoop- ParErr- Stepping- SERR- FastB2B- DisIn
Tx-
x-
Status: Cap+ 66MHz- UDF- FastB2B- ParErr- DEVSEL=fast >TAbort- <TAbort- <MAbort- >SERR- <PERR- INT
Interrupt: pin A routed to IRQ 255
Region 0: Memory at 20200000 (32-bit, non-prefetchable) [disabled] [size=64K]
Region 1: Memory at 20210000 (32-bit, non-prefetchable) [disabled] [size=64K]
Capabilities: [40] Power Management version 3
Flags: PMEClk- DSI- D1- D2- AuxCurrent=0mA PME(D0+,D1+,D2+,D3hot+,D3cold-)
Status: D0 NoSoftRst+ PME-Enable- DSel=0 DScale=0 PME-
Capabilities: [48] MSI: Enable- Count=1/1 Maskable- 64bit+
Address: 0000000000000000 Data: 0000
Capabilities: [60] Express (v2) Endpoint, MSI 00
DevCap: MaxPayload 512 bytes, PhantFunc 0, Latency L0s <64ns, L1 unlimited
ExtTag+ AttnBtn- AttnInd- PwrInd- RBE+ FLReset- SlotPowerLimit 0W
DevCtl: CorrErr- NonFatalErr- FatalErr- UnsupReq-
RlxdOrd+ ExtTag+ PhantFunc- AuxPwr- NoSnoop+
MaxPayload 128 bytes, MaxReadReq 512 bytes
DevSta: CorrErr- NonFatalErr- FatalErr- UnsupReq- AuxPwr- TransPend-
LnkCap: Port #0, Speed 5GT/s, Width x1, ASPM L0s, Exit Latency L0s unlimited
ClockPM- Surprise- LLActRep- BwNot- ASPMOptComp-
LnkCtl: ASPM Disabled; RCB 64 bytes, Disabled- CommClk+
ExtSynch- ClockPM- AutwIdis- BwInt- AutBWInt-
LnkSta: Speed 5GT/s, Width x1
TFERR- Train- SlotClk+ DLActive- BWMgmt- ABWMgmt-
DevCap2: Completion Timeout: Range B, TimeoutDis- NROPrPrP- LTR-
10BitTagComp- 10BitTagReq- OBFF Not Supported, ExtFmt- EETLPPrefix-
EmergencyPowerReduction Not Supported, EmergencyPowerReductionInit-
FRS- TPHComp- ExtTPHComp-
AtomicOpsCap: 32bit- 64bit- 128bitCAS-
DevCtl2: Completion Timeout: 50us to 50ms, TimeoutDis- LTR- 10BitTagReq- OBFF Disabled,
AtomicOpsCtl: ReqEn-
LnkCtl2: Target Link Speed: 5GT/s, EnterCompliance- SpeedDis-
Transmit Margin: Normal Operating Range, EnterModifiedCompliance- ComplianceSOS-
Compliance Preset/De-emphasis: -6dB de-emphasis, 0dB preshoot
LnkSta2: Current De-emphasis Level: -6dB, EqualizationComplete- EqualizationPhase1-
EqualizationPhase2- EqualizationPhase3- LinkEqualizationRequest-
Retimer- 2Retimers- CrosslinkRes: unsupported
Capabilities: [100 v1] Device Serial Number 00-00-00-00-00-00-00-00
lspci: unable to load libkmod resources: error -2
root@RK3576-Tronlong:~#

```

图 62

由上图可知，BYPASS 数据空间起始地址为 0x20210000，大小为 64KByte。

执行如下命令，使能 PCIe 设备。

Target# `echo 1 > /sys/class/pci_bus/0000\:01/device/0000\:01\:00.0/enable`

```

root@RK3576-Tronlong:~# echo 1 > /sys/class/pci_bus/0000\:01/device/0000\:01\:00.0/enable
[ 301.649370] pci 0000:01:00.0: enabling device (0000 -> 0002)
root@RK3576-Tronlong:~#

```

图 63

执行如下命令，分别以字节和半字方式访问内存空间将随机数据先写入 FPGA BRAM，再从 FPGA BRAM 读出，并计算读写速率。

Target# `./devmem_rw -t -a 0x20210000 -s 65536 -o halfword`

```

root@RK3576-Tronlong:~# ./devmem_rw -t -a 0x20210000 -s 65536 -o halfword
Testing...
65536 bytes (0.062 MB) writed, 1.601 ms, 39.038 MB/s
65536 bytes (0.062 MB) read, 49.013 ms, 1.275 MB/s
Test pass!
root@RK3576-Tronlong:~#

```

图 64

因我们的存在，让嵌入式应用更简单

Target# ./devmem_rw -t -a 0x20210000 -s 65536 -o byte

```
root@RK3576-Tronlong:~# ./devmem_rw -t -a 0x20210000 -s 65536 -o byte
Testing...
65536 bytes (0.062 MB) writed, 3.205 ms, 19.501 MB/s
65536 bytes (0.062 MB) read, 97.985 ms, 0.638 MB/s
Test pass!
root@RK3576-Tronlong:~#
```

图 65

串口调试终端打印"Test pass!"信息，表示读写测试通过。本次以半字方式访问内存空间测试的 PCIe 写速率为 39.038MB/s，PCIe 读速率为 1.275MB/s；以字节方式访问内存空间测试的 PCIe 写速率为 19.501MB/s，PCIe 读速率为 0.638MB/s。

备注：本案例仅为验证案例功能。由于案例以字节方式访问内存空间，且传输较少的数据量，因此读写速率测试结果偏低。

1.6.3 案例编译（ARM 端）

请将产品资料“4-软件资料\Demo\base-demos\devmem_rw\arm\src\”目录下的案例源码拷贝至 Ubuntu 工作目录。进入案例源码目录，执行如下命令配置交叉编译工具链，并执行 make 命令进行案例编译。编译完成后，将在当前目录下生成可执行文件。

Host# source /home/tronlong/RK3576/rk3576_linux6.1_release/buildroot/output/rockchip_rk3576/host/environment-setup

Host# make

```
tronlong@tronlong-virtual-machine:~/Demo/base-demos/devmem_rw/arm/src$ ls
devmem_rw.c makefile
tronlong@tronlong-virtual-machine:~/Demo/base-demos/devmem_rw/arm/src$ source /home/tronlong/RK3576/rk3576_linux6.1_release/buildroot/output/rockchip_rk3576/host/environment-setup

  _____
 |  _   _  |
 | | | | | |
 | |_| | | |
 |  _  | | |
 | | | | | |
 |_____|_|_|

Making embedded Linux easy!

Some tips:
* PATH now contains the SDK utilities
* Standard autotools variables (CC, LD, CFLAGS) are exported
* Kernel compilation variables (ARCH, CROSS_COMPILE, KERNELDIR) are exported
* To configure do "./configure $CONFIGURE_FLAGS" or use
  the "configure" alias
* To build CMake-based projects, use the "cmake" alias

tronlong@tronlong-virtual-machine:~/Demo/base-demos/devmem_rw/arm/src$ make
aarch64-buildroot-linux-gnu-gcc -c devmem_rw.c -o devmem_rw.o -I. -Wall -g
aarch64-buildroot-linux-gnu-gcc -o devmem_rw devmem_rw.o -lpthread
tronlong@tronlong-virtual-machine:~/Demo/base-demos/devmem_rw/arm/src$ ls
devmem_rw devmem_rw.c devmem_rw.o makefile
tronlong@tronlong-virtual-machine:~/Demo/base-demos/devmem_rw/arm/src$
```

图 66

因我们的存在，让嵌入式应用更简单

1.6.4 关键代码（ARM 端）

（1）将内存设备地址映射至虚拟地址。

```

391 // input: length, phy_addr
392 // output: fd, map_base, virt_addr
393 int map_addr(int *fd, size_t length, off_t phy_addr, void **map_base, void **virt_addr) {
394     // Open /dev/mem device to map.
395     if ((*fd = open("/dev/mem", O_RDWR | O_SYNC)) == -1) {
396         fprintf(stderr, "Open /dev/mem failed!\n");
397         return -1;
398     }
399
400     // Call mmap to device memory, physics to virtual address.
401     off_t pa_offset = phy_addr & ~(sysconf(_SC_PAGE_SIZE) - 1);
402     *map_base = mmap(NULL, length + phy_addr - pa_offset, PROT_READ | PROT_WRITE, MAP_SHARED, *fd, pa_offset);
403     if (*map_base == (void *) -1) {
404         fprintf(stderr, "mmap failed!\n");
405         close(*fd);
406         return -1;
407     }
408     *virt_addr = *map_base + phy_addr - pa_offset;
409
410     return 0;
411 }

```

图 67

（2）根据指定的操作方式进行数据读取。

```

290 int read_data(void *virt_addr, void *data, int size, char *op_type) {
291     assert(strcmp("byte", op_type) == 0 || strcmp("halfword", op_type) == 0);
292
293     struct timeval time_start, time_stop, time_diff;
294     gettimeofday(&time_start, 0);
295
296     int j;
297     if (strcmp("byte", op_type) == 0) {
298         // 'byte' access operation type, 8 bit at a time.
299         uint8_t *in = (uint8_t *)virt_addr;
300         uint8_t *out = (uint8_t *)data;
301         for (j = 0; j < size; j++) {
302             // This assignment is reading from devie memory, actually.
303             out[j] = in[j];
304         }
305     }
306     else {
307         // 'halfword' access operation type, 16 bit at a time.
308         uint16_t *in = (uint16_t *)virt_addr;
309         uint16_t *out = (uint16_t *)data;
310         for (j = 0; j < size / 2; j++) {
311             // This assignment is reading from devie memory, actually.
312             out[j] = in[j];
313         }
314     }
315
316     // Compute time cost and speed.
317     float time_cos, speed;
318     gettimeofday(&time_stop, 0);
319     timersub(&time_stop, &time_start, &time_diff);
320     time_cos = time_diff.tv_sec * 1000000.0 + time_diff.tv_usec;
321     speed = size / 1024.0 / 1024.0 / (time_cos / 1000000.0);
322     printf("%d bytes (%1.3f MB) read, %1.3f ms, %1.3f MB/s\n",
323         size,
324         size / 1024.0 / 1024.0,
325         time_cos / 1000.0,
326         speed);
327
328     return 0;
329 }

```

图 68

（3）根据指定的操作方式进行数据写入。

因我们的存在，让嵌入式应用更简单

```

331 int write_data(void *virt_addr, void *data, int size, char *op_type) {
332     assert(strcmp("byte", op_type) == 0 || strcmp("halfword", op_type) == 0);
333
334     struct timeval time_start, time_stop, time_diff;
335     gettimeofday(&time_start, 0);
336
337     int i;
338     if (strcmp("byte", op_type) == 0) {
339         // 'byte' access operation type, 8 bit at a time.
340         uint8_t *in = (uint8_t *)virt_addr;
341         uint8_t *out = (uint8_t *)data;
342         for(i = 0; i < size; i++) {
343             // This assignment is writing to devie memory, actually.
344             in[i] = out[i];
345         }
346     }
347     else {
348         // 'halfword' access operation type, 16 bit at a time.
349         uint16_t *in = (uint16_t *)virt_addr;
350         uint16_t *out = (uint16_t *)data;
351         for(i = 0; i < size / 2; i++) {
352             // This assignment is writing to devie memory, actually.
353             in[i] = out[i];
354         }
355     }
356
357     // Compute time cost and speed.
358     float time_cos, speed;
359     gettimeofday(&time_stop, 0);
360     timersub(&time_stop, &time_start, &time_diff);
361     time_cos = time_diff.tv_sec * 1000000.0 + time_diff.tv_usec;
362     speed = size / 1024.0 / 1024.0 / (time_cos / 1000000.0);
363     printf("%d bytes (%1.3f MB) writed, %1.3f ms, %1.3f MB/s\n",
364           size,
365           size / 1024.0 / 1024.0,
366           time_cos / 1000.0,
367           speed);
368
369     return 0;
370 }

```

图 69

2 Python 开发案例

本章节以两个简单案例演示 Python 的使用方法。系统支持 Python3.11，相关库位于文件系统"/usr/lib/python3.11/"目录下。

```

root@RK3576-Tronlong:~# ls /usr/lib/python3.11/
LICENSE.txt      cgitb.py          fractions.py       netrc.py          secrets.py         timeit.py
__future__.py    chunk.py           ftplib.py          nntplib.py        selectors.py       token.py
__hello__.py     cmd.py            funcitools.py     ntpath.py         shelve.py         tokenize.py
__phello__       code.py           genericpath.py    nturl2path.py     shlex.py          tomlib
aix-support.py   codecs.py         getopt.py          numbers.py         signal.py          trace.py
bootsubprocess.py collections.py      getpass.py        opcode.py          site-packages     traceback.py
compat_pickle.py colorsys.py        gettext.py        opcode.py          site.py            tracemalloc.py
compression.py   compileall.py     glob.py           operator.py        smtpd.py           tty.py
markupbase.py    concurrent        graphlib.py       optparse.py        smtplib.py         turtle.py
osx-support.py   config-3.11-aarch64-linux-gnu gzip.py            os.py              sndhdr.py          types.py
py_abc.py        configparser.py   hashlib.py        pdb.py             socket.py           typing.py
pydecimal.py     contextlib.py     heapq.py          pickle.py           socketserver.py    unittest.py
pyio.py          contextvars.py    html.py           pickletools.py     sqlite3            urllib
sitebuiltins.py  copy.py           http.py           pipes.py            sre_compile.py     uu.py
strptime.py      copyreg.py        imaplib.py        pkgutil.py         sre_constants.py   uuid.py
sysconfigdata__linux_aarch64-linux-gnu.py  crypt.py          platform.py        poplib.py           sre_parse.py       venv
threading_local.py weakrefset.py     csv.py            plistlib.py         ssl.py             warnings.py
weakrefset.py    abc.py            dataclasses.py    profile.py          stat.py            wave.py
aifc.py          antigravity.py    dbm.py            pstats.py           string.py           weakref.py
argparse.py      ast.py            decimal.py        psutil.py          stringprep.py      webbrowser.py
asyncio          asyncchat.py      difflib.py        pty.py             struct.py           wsgiref
base64.py        asyncio          dis.py            py_compile.py       subprocess.py       xdrlib.py
bdb.py           base64.py         distutils          pyclbr.py          sunau.py           xml
bisect.py        bz2.py            distutils          pydoc.py            symtable.py        xmlrpc
calendar.py      cgi.py            doctest.py        pydoc.py            sysconfig.py       zipapp.py
cgi.py           cProfile.py       email.py           queue.py            sysconfig.py       zipfile.py
cgitb.py         calendar.py       encodings          quopri.py           tarfile.py         zipimport.py
chunk.py          calender.py       ensurepip          random.py           tempfile.py         zoneinfo
cmd.py            cgi.py            enum.py            re                  textwrap.py        __pycache__
code.py           cgitb.py          filecmp.py        reprlib.py          this.py             __pycache__
codecs.py         chunk.py           fileinput.py      rfc822.py           threading.py
collections.py    compat_pickle.py  fnmatch.py        rfc822.py           __pycache__

```

图 70

因我们的存在，让嵌入式应用更简单

2.1 led_flash 案例

2.1.1 案例功能

本案例通过向评估底板用户可编程指示灯 LED 设备节点反复交替写入 1、0 数值，实现 LED 闪烁效果。LED 点亮与熄灭时间均为 0.5s。

程序流程如下图所示。

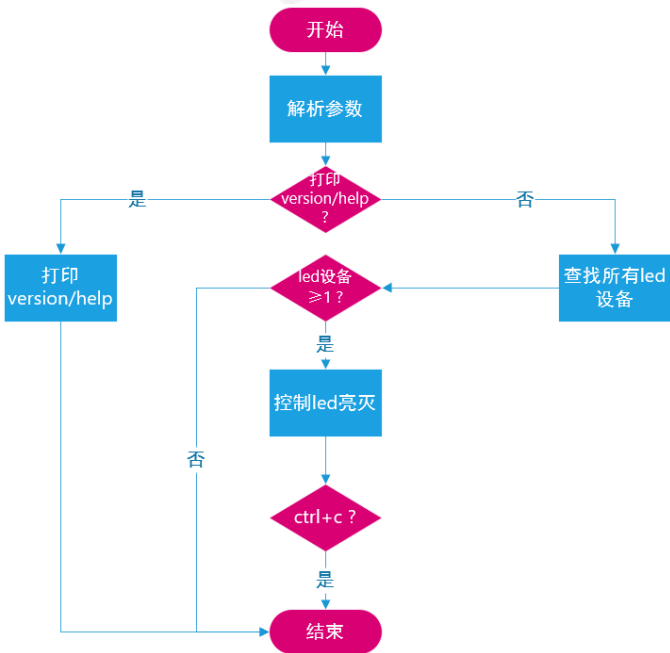


图 71

LED 设备节点为“/sys/class/leds/user-ledX/”目录下的 brightness。

```
root@RK3576-Tronlong:~# ls /sys/class/leds/
disk heartbeat mmc0:: user-led0 user-led1
root@RK3576-Tronlong:~# ls /sys/class/leds/user-led0
brightness device max_brightness power subsystem trigger uevent
root@RK3576-Tronlong:~# ls /sys/class/leds/user-led1
brightness device max_brightness power subsystem trigger uevent
root@RK3576-Tronlong:~#
```

图 72

2.1.2 操作说明

将案例目录下的 led_flash.py 脚本文件拷贝至评估板文件系统，并在脚本文件所在目录执行如下命令查看程序参数信息。

因我们的存在，让嵌入式应用更简单

Target# ./led_flash.py -h

```
root@RK3576-Tronlong:~# ./led_flash.py -h
Usage: ./led_flash.py [options]
Options:
  -v | --version Display version information
  -h | --help    Display help
root@RK3576-Tronlong:~#
```

图 73

执行如下命令运行脚本程序，即可看到评估底板上的 LED 进行闪烁。

Target# ./led_flash.py

```
root@RK3576-Tronlong:~# ./led_flash.py
find leds:
/sys/class/leds/user-led0
/sys/class/leds/user-led1
flash led ...
```

图 74

可按下"Ctrl + C"终止程序。

2.1.3 关键代码

(1) 查找所有 LED 设备。

```
22 """ enumerate all led """
23 def enumerate_led(self):
24     led_name = "user-led"
25
26     """ find led """
27     for filename in os.listdir(self.led_path):
28         if led_name in filename:
29             self.leds.append(os.path.join(self.led_path, filename))
30
31     if len(self.leds) == 0:
32         return False
33
34     """
35     led sort, e.g.
36     /sys/class/leds/user-led0
37     /sys/class/leds/user-led1
38     /sys/class/leds/user-led2
39     """
40     self.leds.sort()
41
42     print ("find leds:")
43     for led in self.leds:
44         print (led)
45
46     return True
```

图 75

因我们的存在，让嵌入式应用更简单

(2) 控制 LED 亮灭。

```
48 """ control led flash """
49 def flash_led(self):
50     led_num = len(self.leds)
51
52     while not Led.quit_flag:
53         """ Turn on leds """
54         for i in range(led_num):
55             """ Set the led brightness value to 1 to turn on the led """
56             ret = os.system("echo 1 > %s/brightness" % (self.leds[i]))
57
58             if ret != 0:
59                 print ("Error: Failed to turn on %s" % self.leds[i])
60
61         """ Keep the leds on for 500 ms """
62         time.sleep(0.5)
63
64         for i in range(led_num):
65             """ Set the led brightness value to 0 to turn off the LED """
66             os.system("echo 0 > %s/brightness" % (self.leds[i]))
67
68         """ Keep the leds off for 500 ms """
69         time.sleep(0.5)
```

图 76

2.2 key_demo 案例

2.2.1 案例功能

本案例通过监听用户输入按键 USER1(KEY4)、USER2(KEY5)的状态，检测按键事件。
程序流程如下图所示。

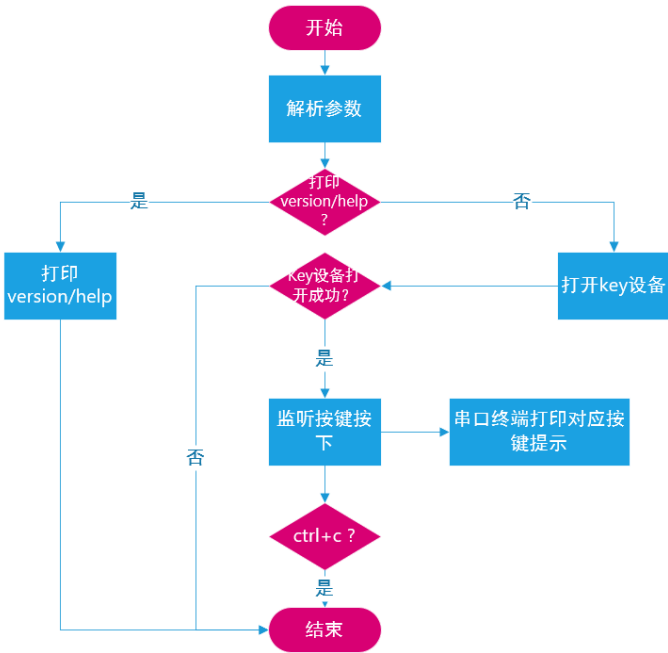


图 77

2.2.2 操作说明

本次测试以用户输入按键 USER1(KEY4)示例进行测试，如需测试用户输入按键 USER2(KEY5)，请修改对应的设备节点。

表 6

用户输入按键	按键事件号
USER1(KEY4)	/dev/input/event5
USER2(KEY5)	/dev/input/event8

执行如下命令，查看 USER1(KEY4)按键对应的设备节点。

Target# cat /proc/bus/input/devices


```
root@RK3576-Tronlong:~# cat /proc/bus/input/devices
I: Bus=0019 Vendor=0000 Product=0000 Version=0000
N: Name="rk805_pwrkey"
P: Phys=rk805_pwrkey/input0
S: Sysfs=/devices/platform/2ac40000.i2c/i2c-1/1-0023/rk805-pwrkey.2.auto/input/input0
U: Uniq=
H: Handlers=kbd event0
B: PROP=0
B: EV=3
B: KEY=10000000000000 0

I: Bus=0019 Vendor=0001 Product=0001 Version=0100
N: Name="headset-keys"
P: Phys=headset-keys/input0
S: Sysfs=/devices/platform/es8388-sound/input/input1
U: Uniq=
H: Handlers=kbd event1
B: PROP=0
B: EV=3
B: KEY=10000000000 0 0
```

图 78

```
I: Bus=0019 Vendor=0001 Product=0001 Version=0100
N: Name="adc-keys2"
P: Phys=adc-keys/input0
S: Sysfs=/devices/platform/adc-keys2/input/input5
U: Uniq=
H: Handlers=kbd event5
B: PROP=0
B: EV=3
B: KEY=1000000 0 0

I: Bus=0000 Vendor=0000 Product=0000 Version=0000
N: Name="rockchip-dp0 rockchip-dp0"
P: Phys=ALSA
S: Sysfs=/devices/platform/dp0-sound/sound/card1/input6
U: Uniq=
H: Handlers=event6
B: PROP=0
B: EV=21
B: SW=40
```

图 79

将案例目录下的 key_demo.py 脚本文件拷贝至评估板文件系统，并在脚本文件所在目录执行如下命令查看程序参数信息。

Target# ./key_demo.py -h

```
root@RK3576-Tronlong:~# ./key_demo.py -h
usage: ./key_demo.py [options]
options:
  -d | --device Device name (such as: /dev/input/event0)
  -v | --version Display version information
  -h | --help    Display help

e.g. :
    ./key_demo.py -d /dev/input/event0
root@RK3576-Tronlong:~#
```

图 80

执行如下命令运行脚本程序，串口终端将打印提示信息。按下评估板用户输入按键 USER1(KEY4)，程序将检测到按键事件，并打印按键状态信息。

备注：本次测试以用户输入按键 USER1(KEY4)示例进行测试，如需测试用户输入按键

因我们的存在，让嵌入式应用更简单

USER2(KEY5)，请修改为对应的按键事件号。

Target# ./key_demo.py -d /dev/input/event5

```
root@RK3576-Tronlong:~# ./key_demo.py -d /dev/input/event5
Please press the key to test.
User key pressed!
User key pressed!
User key pressed!
```

图 81

可按下"Ctrl + C"终止程序。

2.2.3 关键代码

(1) 打开按键设备。

```
23 def open_device(self, key_dev):
24     try:
25         """ open key device, if success, return True"""
26         self.key_dev = evdev.InputDevice(key_dev)
27         return True
28     except Exception as e:
29         print (e)
30         return False
```

图 82

(2) 监听按键事件。

```
32 def listen_key_pressed(self):
33     try:
34         print ("Please press the key to test.\n")
35
36         """
37         Listen for the button to be pressed
38         event.value: 1: pressed; 0: released
39         """
40         while not KeyDevice.quit_flag:
41             event = self.key_dev.read_one()
42             if not event:
43                 continue
44
45             if event.type == evdev.ecodes.EV_KEY:
46                 if event.code == evdev.ecodes.KEY_PROG1 and event.value == 1:
47                     print ("User key pressed!\n")
48
49                 elif event.code == evdev.ecodes.KEY_PROG2 and event.value == 1:
50                     print ("User key pressed!\n")
51
52         except Exception as e:
53             if not KeyDevice.quit_flag:
54                 print (e)
55
56     @classmethod
57     def stop(cls):
58         cls.quit_flag = True
```

图 83

更多帮助

销售邮箱: sales@tronlong.com

技术邮箱: support@tronlong.com

创龙总机: 020-8998-6280

技术热线: 020-3893-9734

创龙官网: www.tronlong.com

技术论坛: www.51ele.net

官方商城: tronlong.tmall.com